

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	S.S.Janane
Register Number	3122247001024
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20-9-2026

1 Objective

The goal of this experiment is to compare a Single Layer Perceptron (PLA) and a Multilayer Perceptron (MLP) on a handwritten character recognition task. Model A is a PLA built from scratch. Model B is an MLP tuned using grid search over activation, optimizer, learning rate, hidden layers, and batch size.

2 Problem Statement

Input: a 32 by 32 grayscale image of a handwritten character, flattened into 1024 pixel values and normalized.

Output: the predicted character class, one of 62 classes (0-9, A-Z, a-z).

Objective: train PLA and MLP, tune MLP hyperparameters, and compare both models on accuracy, precision, recall, F1, ROC curves, and confusion matrices.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Number of Images	3410
Number of Classes	62 (digits 0-9, uppercase A-Z, lowercase a-z)
Images per Class	55
Image Size Used	32 x 32, grayscale, flattened to 1024 features
Train-Test Split	80 percent train (2728 images), 20 percent test (682 images), stratified

4 Exploratory Data Analysis

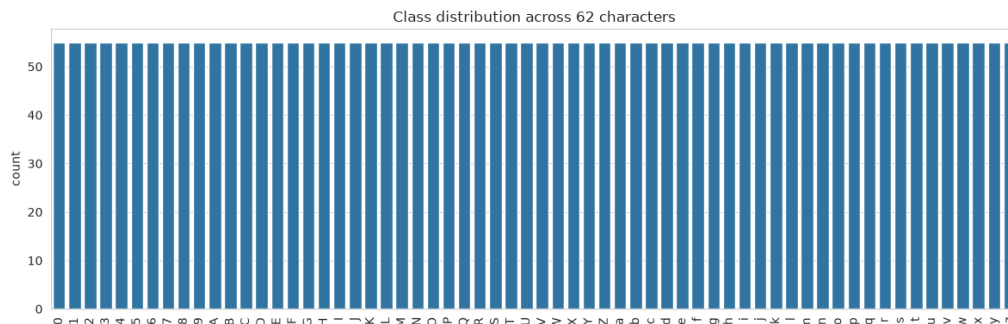


Figure 1: Class distribution across the 62 characters

Inference:

Every class has exactly 55 images. The dataset is perfectly balanced. This makes accuracy and macro averaged metrics fair to compare across models.

5 Data Preprocessing

- Images were converted to grayscale.
- Images were resized to 32 by 32 pixels.
- Pixel values were flattened into a 1024 length vector and normalized to the range 0 to 1.
- Labels were encoded from characters into integers using a label encoder.
- Data was split 80 percent train, 20 percent test, using stratified sampling so every class appears in both sets.

6 Machine Learning Algorithm

Model A: Single Layer Perceptron (PLA)

Working principle: PLA uses a step activation function. For multiple classes, one perceptron is trained per class using a one vs rest approach. Each perceptron learns a linear boundary that separates its class from all others.

Weight update rule:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

Limitation: PLA only works well when classes are linearly separable. Handwritten characters have overlapping, curved, and nonlinear shapes, so a single straight line boundary is often not enough.

Model B: Multilayer Perceptron (MLP)

Architecture: Input layer, one or more hidden layers, output layer.

Activation functions tried: ReLU, Tanh.

Loss function: Cross entropy loss for multi class classification.

Optimizers tried: SGD, Adam.

Working principle: MLP passes input through hidden layers with nonlinear activations, then uses backpropagation to update all weights based on the error at the output layer. This lets it learn nonlinear decision boundaries, unlike PLA.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU, no GPU used

8 Hyperparameter Selection

The MLP was tuned using grid search with 3 fold cross validation over the following settings.

Hyperparameter	Values Tried
Hidden layer sizes	(64), (128, 64), (128, 64, 32)
Activation	relu, tanh
Solver (optimizer)	sgd, adam
Learning rate init	0.001, 0.01
Batch size	32, 64

Best configuration found:

Hyperparameter	Best Value
Hidden layer sizes	(128, 64)
Activation	tanh
Solver	sgd
Learning rate init	0.01
Batch size	64
Best cross validation accuracy	0.3644

Justification: this configuration gave the highest cross validation accuracy among all 24 combinations tried. Two hidden layers gave the model enough capacity to learn nonlinear pixel patterns without being as slow or as prone to overfitting as the three layer option.

9 Results

Metric	PLA	MLP
Accuracy	0.1026	0.4340
Precision (macro)	0.1841	0.4870
Recall (macro)	0.1026	0.4340
F1 Score (macro)	0.0836	0.4286
ROC AUC (micro)	0.7810	0.9256
ROC AUC (macro)	0.8446	0.9249

PLA finished training with 4209 total misclassifications in its last epoch. MLP converged in 64 iterations with early stopping.

10 Result Visualization

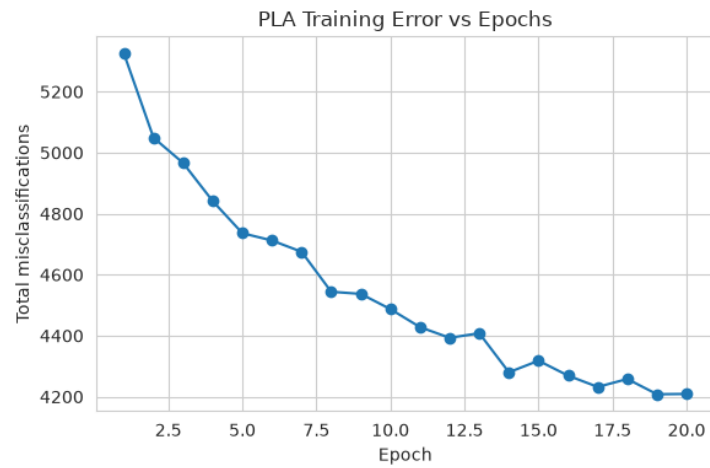


Figure 2: PLA training error versus epochs

Inference:

The number of misclassifications drops steadily as epochs increase. The drop slows down after about epoch 10. This shows PLA is learning, but it plateaus at a high error since the data is not linearly separable.

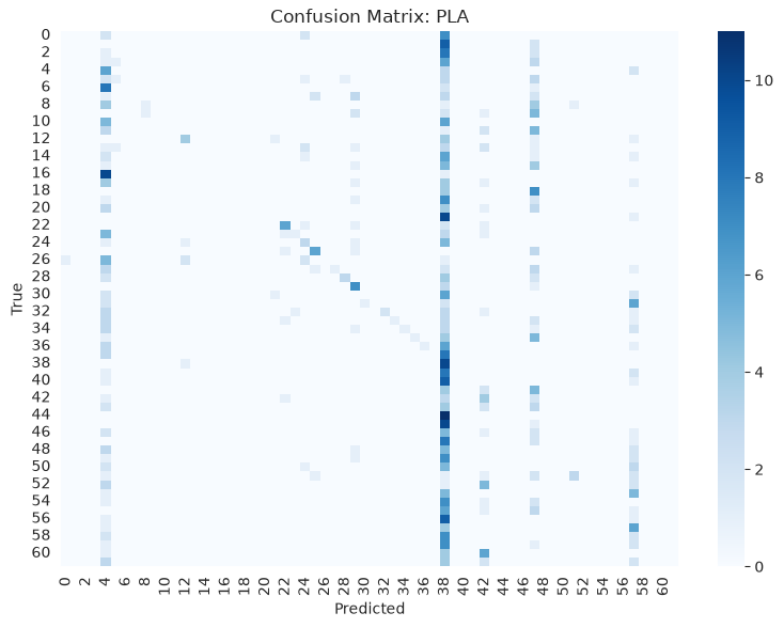


Figure 3: Confusion matrix for PLA

Inference:

The diagonal is faint and scattered, meaning correct predictions are rare. Predictions are spread across many wrong classes. This matches the very low PLA accuracy of about 10 percent.

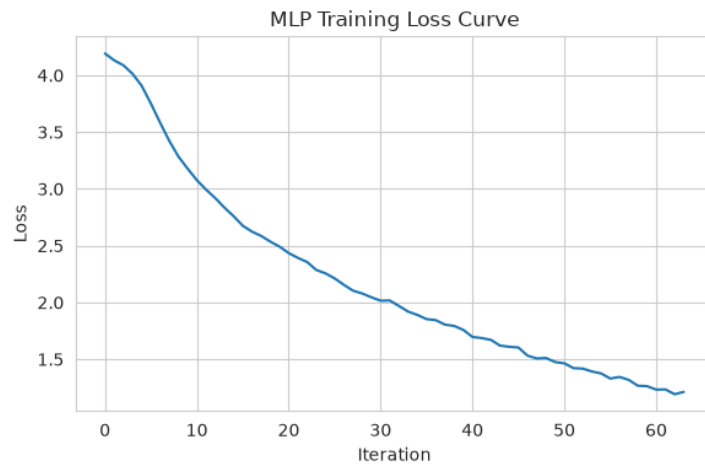


Figure 4: MLP training loss curve

Inference:

The loss drops smoothly and consistently across iterations. There is no sudden jump or spike. This shows stable convergence with the chosen learning rate and optimizer.

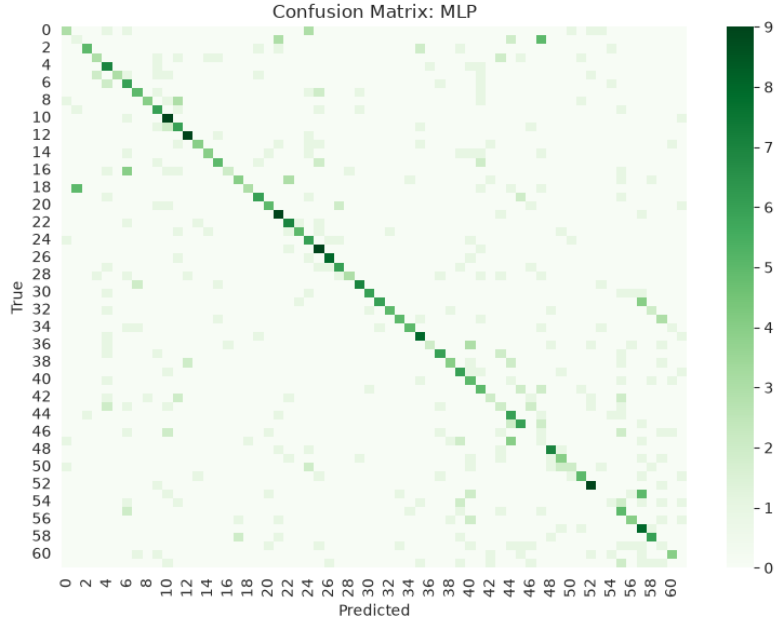


Figure 5: Confusion matrix for MLP

Inference:

The diagonal is much clearer than in the PLA confusion matrix. Most samples are predicted close to their true class. Some confusion still exists between visually similar characters, such as digits and letters that look alike.

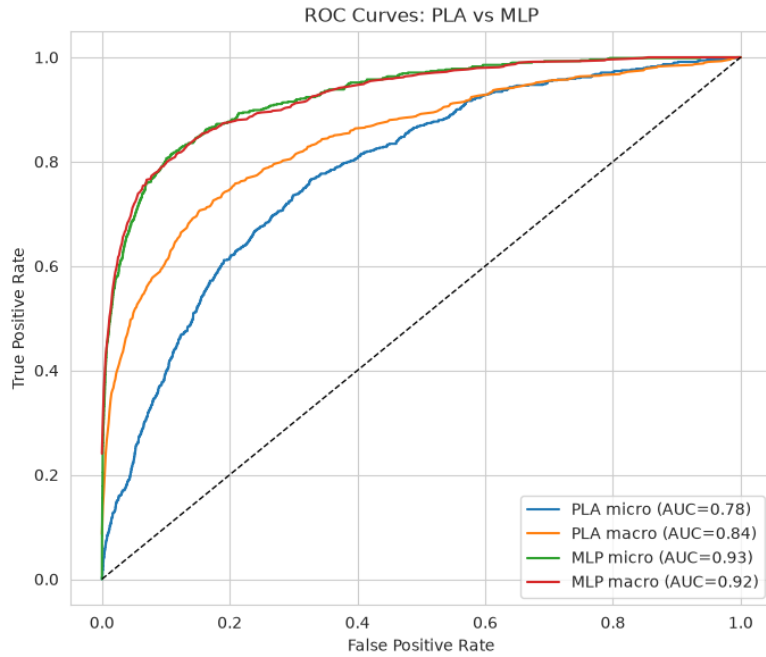


Figure 6: ROC curves for PLA and MLP, micro and macro average

Inference:

MLP curves sit much closer to the top left corner than PLA curves. MLP has a higher area under the curve for both micro and macro average. This confirms MLP separates classes better across all decision thresholds, not just at the default threshold.

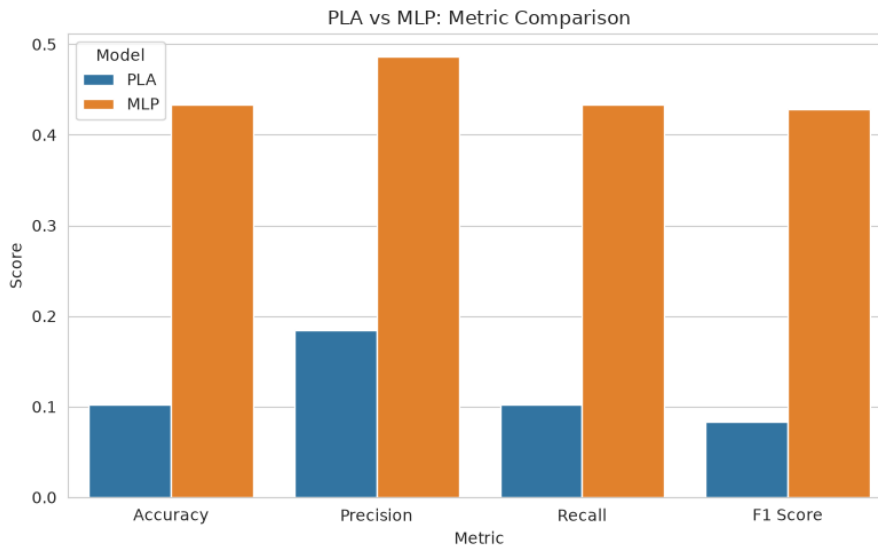


Figure 7: Bar chart comparing PLA and MLP across metrics

Inference:

MLP scores higher than PLA on every metric shown. The gap is largest for F1 score and smallest for precision. This gives a clear, consistent picture that MLP outperforms PLA on this dataset.

11 A/B Comparison: PLA vs MLP

- Final hyperparameters used for MLP: hidden layers (128, 64), tanh activation, sgd optimizer, learning rate 0.01, batch size 64.
- Strength of PLA: simple, fast to implement, easy to understand the weight update rule.
- Weakness of PLA: can only draw straight line boundaries, so it struggles badly with the curved and overlapping shapes of handwritten characters.
- Strength of MLP: hidden layers and nonlinear activation let it learn curved decision boundaries, giving much higher accuracy and ROC AUC.
- Weakness of MLP: needs more tuning, more training time, and more data to avoid overfitting.
- Impact of tuning: without tuning, an MLP with poor settings can converge slowly or get stuck. Grid search found a setting that reached the best cross validation accuracy among all options tried.

12 Observations and Analysis

Why does PLA underperform compared to MLP?

PLA can only separate classes using a straight line boundary for each class. Handwritten characters have curved, overlapping shapes that are not linearly separable. MLP

uses hidden layers and nonlinear activation to draw curved boundaries, so it fits this data much better.

Which hyperparameters had the most impact on MLP performance?

The hidden layer size and the activation function had the biggest impact. Moving from a single hidden layer to two hidden layers, and using tanh instead of relu, gave a noticeably better cross validation score in the grid search results.

Did optimizer choice affect convergence?

Yes. The best model used sgd with a learning rate of 0.01. Adam adapts its own learning rate automatically, so it was less sensitive to the learning rate setting, while sgd needed a higher learning rate to converge well in a reasonable number of iterations.

Did adding more hidden layers always improve results?

No. The three hidden layer option, (128, 64, 32), did not win the grid search. Two hidden layers gave the best cross validation accuracy. Adding another layer likely made training harder to converge well given the small dataset size.

Did MLP show overfitting? How could it be mitigated?

With only 44 training images per class, some overfitting is likely, since the model has many parameters relative to the amount of data. Early stopping was already used to reduce this. Overfitting could be further reduced with more training data, data augmentation, dropout style regularization, or reducing the number of hidden units.

13 Conclusion

The MLP clearly outperformed the PLA on this handwritten character recognition task, reaching 43.4 percent accuracy compared to 10.3 percent for PLA, with a much higher ROC AUC as well. This confirms that PLA is not suitable for tasks with non linear, overlapping classes like handwritten characters, while a tuned MLP with two hidden layers, tanh activation, and sgd optimizer can learn a much better decision boundary. The main limitation across both models is the small number of training images per class, which likely limits how high accuracy can go without more data or a convolutional approach.

14 References

- [1] T. E. de Campos, B. R. Babu, and M. Varma, “Character recognition in natural images,” in Proc. Int. Conf. on Computer Vision Theory and Applications (VISAPP), Lisbon, Portugal, 2009.
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [3] Kaggle, “English Handwritten Characters Dataset,” kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset.